IERG4320/IEMS5726 — Chapter 5: Data Representation III — Cheat Sheet

Prof. YAN Wenjing, Dr. Danny Yip | Department of Information Engineering, CUHK

Artificial Neural Network (ANN/NN)

受人脑启发的计算模型，由神经元(节点)组成，能学习非线性关系。

Neuron (神经元)

基本单元：每个输入乘以权重，加起来，再过一个激活函数f。

$$\text{output} = f\left(\sum_i w_i x_i + b\right)$$

Activation Functions (激活函数)

非线性函数，让NN能学非线性关系。常见：ReLU, Sigmoid, Tanh

Feed-forward NN (前馈神经网络)

- **Input nodes** (输入层): pass data in, no computation
- **Hidden nodes** (隐藏层): do computation
- **Output nodes** (输出层): produce final result

MLP = 有1+个隐藏层的NN = 全连接网络（每个节点连下一层所有节点）

Key Parameters (关键参数)

- # layers, # nodes per layer (层数、每层节点数)
- Activation function (激活函数)
- Optimizer (优化器，如SGD/Adam)
- Learning rate (学习率，控制步长)
- Batch size (每次喂多少数据)
- Epoch (整个数据集过几遍)

Loss Function (损失函数)

衡量预测和真实值的差距。分类常用Log Loss: $\text{LogLoss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log(p_{ic})$

N=样本数, C=类别数, y_{ic}=是否属于类c, p_{ic}=预测概率

Training Steps (训练步骤)

1. Pick architecture (选网络结构)
2. Random init weights (随机初始化权重)
3. Forward propagation (前向传播：输入→预测)
4. Compute loss (算损失)
5. Backpropagation (反向传播：算梯度)
6. Gradient descent (梯度下降：更新权重)
7. Repeat until converge (重复直到收敛)

Simple ANN in PyTorch (代码)

```python
import torch
import torch.nn as nn
n_input, n_hidden, n_out = 10, 15, 1
batch_size, lr = 100, 0.01
# Random dummy data
data_x = torch.randn(batch_size, n_input)
data_y = (torch.rand(batch_size, 1) < 0.5).float()

# Define model
model = nn.Sequential(
    nn.Linear(n_input, n_hidden), nn.ReLU(),
    nn.Linear(n_hidden, n_out), nn.Sigmoid())
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(
    model.parameters(), lr=lr)
```

Training Loop (训练循环)

```python
losses = []
for epoch in range(50000):
    pred_y = model(data_x)        # Forward
    loss = loss_fn(pred_y, data_y)
    losses.append(loss.item())
    model.zero_grad()
    loss.backward()               # Backward
    optimizer.step()              # Update
# Plot loss curve
import matplotlib.pyplot as plt
plt.plot(losses); plt.ylabel('loss')
plt.xlabel('epoch'); plt.show()
```

ANN Limitations (ANN局限)

- Prone to overfitting (容易过拟合)
- Sensitive to outliers (对异常值敏感)
- Data-driven, needs lots of data (依赖大量数据)
- Issues with imbalanced classes (类别不平衡时有问题)

Autoencoder (自编码器)

无监督NN，输入=输出。目标是学到一个压缩的中间表示（latent space）。

Encoder: 高维输入 → 低维编码（压缩）
Bottleneck: 最小的中间层（潜在空间）
Decoder: 低维编码 → 重建原始输入（解压缩）
类比：PCA的非线性版本

Use Cases (应用场景)

- Data compression (数据压缩)
- Denoising (去噪)
- Anomaly detection (异常检测)
- Visualization (降维可视化)
- Feature engineering (特征提取)
- Generation (VAE) (数据生成，如DALL-E)

MNIST Autoencoder Code (完整代码)

```python
# Load MNIST dataset
import torch
from torch import nn, optim
from torchvision import datasets, transforms
transform = transforms.ToTensor()
dataset = datasets.MNIST(root="./data",
    train=True, download=True,
    transform=transform)
loader = torch.utils.data.DataLoader(
    dataset=dataset, batch_size=32, shuffle=True)
```

```python
# Define Autoencoder
class AE(nn.Module):
    def __init__(self):
        super(AE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(28*28, 128), nn.ReLU(),
            nn.Linear(128, 64),    nn.ReLU(),
            nn.Linear(64, 36),     nn.ReLU(),
            nn.Linear(36, 18),     nn.ReLU(),
            nn.Linear(18, 9))      # bottleneck=9
        self.decoder = nn.Sequential(
            nn.Linear(9, 18),      nn.ReLU(),
            nn.Linear(18, 36),     nn.ReLU(),
            nn.Linear(36, 64),     nn.ReLU(),
            nn.Linear(64, 128),    nn.ReLU(),
            nn.Linear(128, 28*28),nn.Sigmoid())
    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded, encoded
```

encoder: 784→128→64→36→18→9（压缩）

decoder: 9→18→36→64→128→784（还原）

```python
# Train Autoencoder
model = AE()
loss_fn = nn.MSELoss()  # reconstruction loss
optimizer = optim.Adam(model.parameters(),
    lr=1e-3, weight_decay=1e-8)
device = torch.device(
    "cuda" if torch.cuda.is_available()
    else "cpu")
model.to(device)
for epoch in range(20):
    for images, _ in loader:
        images = images.view(-1, 28*28).to(device)
        reconstructed = model(images)[0]
        loss = loss_fn(reconstructed, images)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch+1}/20, "
          f"Loss: {loss.item():.6f}")
```

MSE Loss = 原图和重建图的像素差平方和

Latent Space Size (怎么定bottleneck大小？)

1. Domain knowledge (领域知识，如MNIST用9或10)
2. Progressive reduction (试：调大调小看效果)
3. PCA variance analysis (用PCA的方差比估计)
4. Rule of thumb: 2^n for images (经验法则)

Limitations (局限)

- Needs enough training data (数据不够学不好)
- Lossy compression (压缩有损，信息丢失)
- Hard to choose bottleneck size (中间层大小难定)
- Imperfect reconstruction (还原不完美)

Transformer (变换器)

2017年Google提出（"Attention Is All You Need"），专门处理序列数据（文本等），核心是自注意力机制。

Autoencoder vs Transformer

	Autoencoder	Transformer
Purpose	Unsupervised (压缩)	Seq model (序列)
Core	Bottleneck (瓶颈层)	Self-attention (自注意力)
Input	Static (图片等)	Sequential (文本等)
Output	Reconstruction (重建)	Generation (翻译/摘要)
Focus	Data itself (数据)	Context (上下文关系)

Architecture (结构)

- **Encoder** (left): 6 layers of self-attention + FFN
  每层：自注意力 + 前馈网络
- **Decoder** (right): 6 layers of self-attn + cross-attn + FFN
  每层：自注意力 + 编码器-解码器注意力 + 前馈网络

Key Concepts (核心概念)

1. Positional Encoding (位置编码)

Transformer不像RNN有顺序，所以要给每个词加位置信息。

位置向量 + 词向量 = 最终输入

2. Self-Attention (自注意力)

让每个词都能"看到"句子中所有其他词，计算它们的相关性。

Q(Query) × K(Key) = 权重 → 加权求和V(Value)

"bank"旁边是"river"和"account"时会学到不同含义。

3. Cross-Attention (交叉注意力)

解码器查看编码器的输出，Q来自解码器，K/V来自编码器。

4. Multi-Head Attention (多头注意力)

同时用多组Q/K/V，从不同角度捕捉关系。

Data Flow (数据流转)

Words → Token IDs (整数) → Token Embeddings (向量) → + Positional Encoding → Self-Attention → Contextualized Embeddings
最终每个词变成一个蕴含上下文信息的向量。

Use Pre-trained Model! (用预训练模型！)

从头训练非常贵（BERT训练3B词花8天TPU v3）。

微调(fine-tune)只需2–3小时。所以一定要用预训练模型。

```
import transformers
name = "bert-base-uncased"
tokenizer = transformers.AutoTokenizer \
    .from_pretrained(name)
transformer = transformers.AutoModel \
    .from_pretrained(name)
```

Transformer Class (分类模型)

```
class Transformer(nn.Module):
    def __init__(self, transformer,
                 output_dim, freeze):
        super().__init__()
        self.transformer = transformer
        hidden = transformer.config.hidden_size
        self.fc = nn.Linear(hidden, output_dim)
        if freeze:  # freeze pre-trained weights
            for p in self.transformer.parameters():
                p.requires_grad = False
    def forward(self, ids):
        output = self.transformer(
            ids, output_attentions=True)
        hidden = output.last_hidden_state
        attention = output.attentions[-1]
        cls_hidden = hidden[:, 0, :]  # [CLS]
        prediction = self.fc(
            torch.tanh(cls_hidden))
        return prediction
```

hidden[:,0,:] = [CLS] token的向量，代表整句话的含义

BERT Embeddings (提取BERT嵌入)

```
from transformers import (
    BertTokenizer, BertModel)
tokenizer = BertTokenizer.from_pretrained(
    'bert-base-uncased')
model = BertModel.from_pretrained(
    'bert-base-uncased')
model.eval()
```

```
text = "Hello, how are you?"
inputs = tokenizer(text, return_tensors='pt')
with torch.no_grad():
    outputs = model(**inputs)
# Contextualized embeddings
last_hidden = outputs.last_hidden_state
# Shape: (1, seq_len, 768)
```

每个token变成768维向量，包含上下文信息

Transformer Limitations (局限)

- Very expensive to train (训练超费钱)
- Needs good pre-trained model (依赖预训练模型质量)
- Sequence length limits (输入长度有上限)
- Slow generation (生成速度慢)

Google Colab (谷歌Colab)

免费在线Jupyter Notebook，可用GPU/TPU，适合跑深度学习。

- Free GPU/TPU access (免费GPU)
- Pre-installed: PyTorch, TF, Pandas (常用库已装好)
- Save to Google Drive (存到云盘)
- Runtime → Change runtime type → GPU (切换GPU)

Quick Reference (速查总结)

Topic	Key Point
ANN	`nn.Sequential(Linear, ReLU, ...)` (搭积木)
Training	Forward → Loss → Backward → Step (四步循环)
Autoencoder	Encoder→Bottleneck→Decoder (压缩再还原)
Latent space	encoded = model.encoder(x) 提取
Transformer	Self-attention on sequences (序列自注意力)
Pre-trained	`AutoModel.from_pretrained()` (用预训练!)
BERT embed	`outputs.last_hidden_state` (提取嵌入)

核心思路：ANN能学非线性关系 → Autoencoder用ANN做无监督压缩 → Transformer用注意力机制理解序列上下文。现在都用预训练模型微调！